PRINCIPAL AI/ML ENGINEER

Interview Cheatsheet - Expanded

Decision Automation • Experimentation • Causal Inference • Contextual Bandits • Bayesian Skill Models • ML Metrics • Code

Formula-first reference with Python snippets for interview preparation

1. Decision Automation - Mental Model

Core separation - State → features → candidates → policies/models + rules → arbitration → action → logging/feedback. Candidate generation asks “what is allowed?”; the policy asks “which allowed action is best?”

Player/context state

Raw and semi-processed player/session information: progression, recent events, economy, device, locale, attempts, failures, session state.

Feature/state layer

Transform telemetry into numerical/categorical variables. Learned embeddings are optional, not mandatory.

Candidate generation

Produce only feasible actions after eligibility/unlock constraints.

ML policies/models + product rules

Score/select among candidates. Contextual bandits live here.

Central arbitration/orchestration

Resolve conflicts, priorities, budgets, exclusions, global objectives and safety constraints.

Decision + logging + feedback

Execute and log context, candidates, action, policy/model version, propensity, reward, experiment, latency and errors.

Interview answer - “I would retrieve player state, derive features, generate feasible actions, score them with ML policies and product rules, arbitrate conflicts under global constraints, execute the action and log the full decision for experimentation and learning.”

2. Contextual Bandits

Definition - Observe context x, choose an action a, observe reward r, update the policy. It is a lightweight RL setting because there is no explicit long-horizon state-transition problem.

- Exploration = deliberately try uncertain actions to learn.
- Exploitation = choose the action currently estimated to have highest reward.
- Regret = reward lost versus the optimal action/policy; exact per-decision regret is usually unobservable because the counterfactual is missing.

LinUCB - actual equations

$$\hat{\theta}_a = A_a^{-1} b_a$$

$$\hat{r}(x,a) = x^\top \hat{\theta}_a$$

$$UCB(x,a) = x^\top \hat{\theta}_a + \alpha \sqrt{(x^\top A_a^{-1} x)}$$

- x = player/context feature vector; $\hat{\theta}_a$ = learned coefficients for action a; α controls exploration.
- “Linear” means expected reward is a weighted sum of the context features for each action.

Python - contextualbandits / LinUCB

```
import numpy as np
from contextualbandits.online import LinUCB

# X: [n_samples, n_features]
# a: chosen action index for each sample
# r: observed binary reward (0/1)
policy = LinUCB(
    nchoices=3,
    alpha=0.5,
    beta_prior="auto",
    random_state=42,
)
policy.fit(X, a, r)

context = np.array([[0.72, 0.30, 0.65, 0.10]])
action = policy.predict(context)[0]
print("chosen action:", action)
```

Production logging - Always log context, candidate set, selected action, selection probability/propensity, reward/outcome and

policy version. Without propensities, robust off-policy evaluation becomes harder.

3. A/B Testing - From Experiment to Statistics

Control vs treatment - Control = current/base experience. Treatment = new feature/action. Metric = D1/D7 retention, conversion, etc.

- Use stable random assignment by user_id + experiment_id so a user does not jump between variants.
- Track exposure, not just assignment.
- Choose sample size from baseline metric, minimum detectable effect (MDE), α and desired power.

Core formulas for two proportions

$$\hat{\Delta} = \hat{p}^B - \hat{p}^A$$

$$SE(\hat{p}) = \sqrt{\hat{p}(1-\hat{p})/n}$$

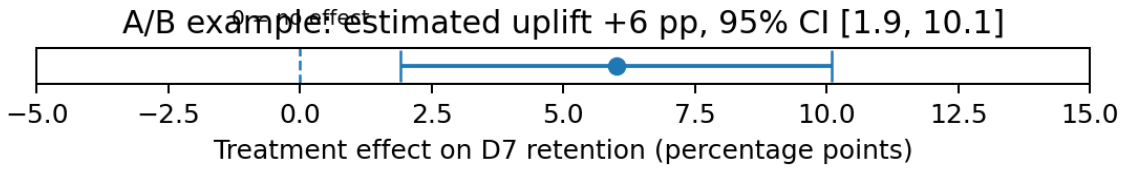
$$SE(\hat{\Delta}) = \sqrt{\hat{p}^A(1-\hat{p}^A)/n^A + \hat{p}^B(1-\hat{p}^B)/n^B}$$

$$z = \hat{\Delta} / SE(\hat{\Delta})$$

$$CI_{95\%} = \hat{\Delta} \pm 1.96 \cdot SE(\hat{\Delta})$$

$$p\text{-value} = 2 \cdot [1 - \Phi(|z|)]$$

- Example: A=30%, B=36%, nA=nB=1000 → $\hat{\Delta}$=+6 pp, SE($\hat{\Delta}$)≈2.1 pp, z≈2.86, p≈0.004, CI≈[1.9,10.1] pp.



If the 95% CI for the treatment effect does not include 0, the observed difference is statistically detectable at the 5% two-sided level.

Python - statsmodels: z-test + 95% CI

```
import numpy as np
from statsmodels.stats.proportion import proportions_ztest
from scipy.stats import norm

count = np.array([300, 360])     # retained in A, B
nobs  = np.array([1000, 1000])

z, p_value = proportions_ztest(count, nobs)

pA, pB = count / nobs
delta = pB - pA
se = np.sqrt(pA*(1-pA)/nobs[0] + pB*(1-pB)/nobs[1])
ci95 = (delta - 1.96*se, delta + 1.96*se)

print(z, p_value, delta, ci95)
```

Python - sample-size / power calculation

```
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize

p_control = 0.30
p_treatment = 0.36
h = proportion_effectsize(p_control, p_treatment)

n_per_group = NormalIndPower().solve_power(
    effect_size=h,
    alpha=0.05,
    power=0.80,
    ratio=1.0,
    alternative="two-sided",
)
print(n_per_group)
```

Python - small/sparse samples: Fisher exact

```
from scipy.stats import fisher_exact

# rows=A/B, columns=retained/not retained
table = [[15, 35],
         [18, 32]]

odds_ratio, p_value = fisher_exact(table, alternative="two-sided")
```

Normal approximation rule of thumb - For each group, require roughly $n\cdot\hat{p} \geq 10$ and $n\cdot(1-\hat{p}) \geq 10$ before casually relying on a normal approximation. This is a heuristic, not a law.

4. Causal Inference, Uplift and Counterfactuals

Prediction vs intervention - Prediction asks P(Y|X): what is likely to happen? Causal inference asks what would change under alternative interventions.

$$\mathbf{ATE = E[Y(1) - Y(0)]}$$

$$\mathbf{CATE(x) = E[Y(1) - Y(0) \mid X=x]}$$

$$\mathbf{ITE_i = Y_i(1) - Y_i(0)}$$

- ATE = average effect across the population.
- CATE = average effect for a context/segment.
- ITE = conceptual individual effect; in reality both potential outcomes are never observed for the same person at the same time.
- Uplift modeling estimates incremental causal benefit of treatment, often at segment/person level.

Propensity score - e(x)=P(T=1|X=x): probability of receiving treatment given observed covariates. Useful when treatment was not randomized; matching/weighting attempts to make observational groups comparable.

- If you already have a clean randomized A/B test, propensity-score correction is generally unnecessary.

5. Offline Policy Evaluation (OPE)

Problem - Historical logs contain reward only for the action actually taken. The reward for unchosen actions is counterfactual and missing.

- Replay: evaluate only logged rows where the candidate policy agrees with the historical action. Simple but wastes data.
- IPS: reweight observed rewards by candidate-policy probability divided by logging-policy propensity.

$$\mathbf{\hat{V}^{IPS}(\pi) = (1/N)\ \Sigma_i\ [\ \pi(a_i|x_i)\ /\ \mu(a_i|x_i)\ ]\ r_i}$$

- Doubly Robust: combine propensity weighting with a learned reward model to reduce sensitivity to one imperfect component.

$$\mathbf{\hat{V}^{DR} = (1/N)\ \Sigma_i\ \{\ \hat{q}(x_i,\pi) + [\pi(a_i|x_i)/\mu(a_i|x_i)]\cdot[r_i - \hat{q}(x_i,a_i)]\ \}}$$

Release sequence - OPE → shadow → canary → controlled online A/B → progressive rollout.

6. Bayesian Skill vs Level Difficulty

Bayesian update - Maintain uncertainty over latent quantities such as player skill. Prior + likelihood → posterior; the posterior becomes the next prior as evidence arrives.

$$\mathbf{p(\theta \mid D) = p(D \mid \theta)\cdot p(\theta) / p(D)}$$

$$\mathbf{P(Y_{ij}=1) = \sigma(\theta_i - \beta_j) = 1 / [1 + e^{-(\theta^i - \beta^j)}]}$$

- θ_i = latent skill of player i; β_j = latent difficulty of level j.
- Rasch / 1PL IRT estimates both jointly, so you do not need to know one perfectly before estimating the other.
- Priors anchor the model and provide sensible cold-start uncertainty. Hierarchical priors can share information among similar players/levels.

Python - PyMC Bayesian Rasch / 1PL model

```
import pymc as pm
import numpy as np

# player_idx[k], level_idx[k], success[k] describe attempt k
n_players = int(player_idx.max()) + 1
n_levels  = int(level_idx.max()) + 1

with pm.Model() as model:
    skill = pm.Normal("skill", mu=0.0, sigma=1.0, shape=n_players)
    difficulty = pm.Normal("difficulty", mu=0.0, sigma=1.0, shape=n_levels)

    logit_p = skill[player_idx] - difficulty[level_idx]
    p_success = pm.Deterministic("p_success", pm.math.sigmoid(logit_p))

    y = pm.Bernoulli("y", p=p_success, observed=success)
    trace = pm.sample(1000, tune=1000, target_accept=0.9)

# Posterior mean skill estimate for each player
skill_hat = trace.posterior["skill"].mean(("chain", "draw")).values
```

Interview answer - "I would model player ability and level difficulty jointly with a Bayesian IRT/Rasch model, retaining

posterior uncertainty for cold start and continuously updating as attempts arrive.”

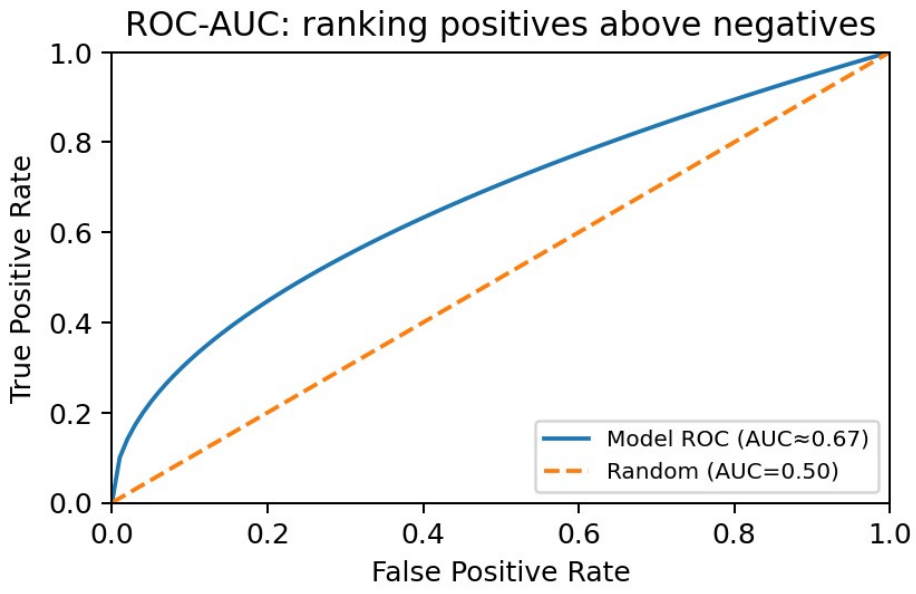
7. ML Metrics - Which Metric for Which Problem?

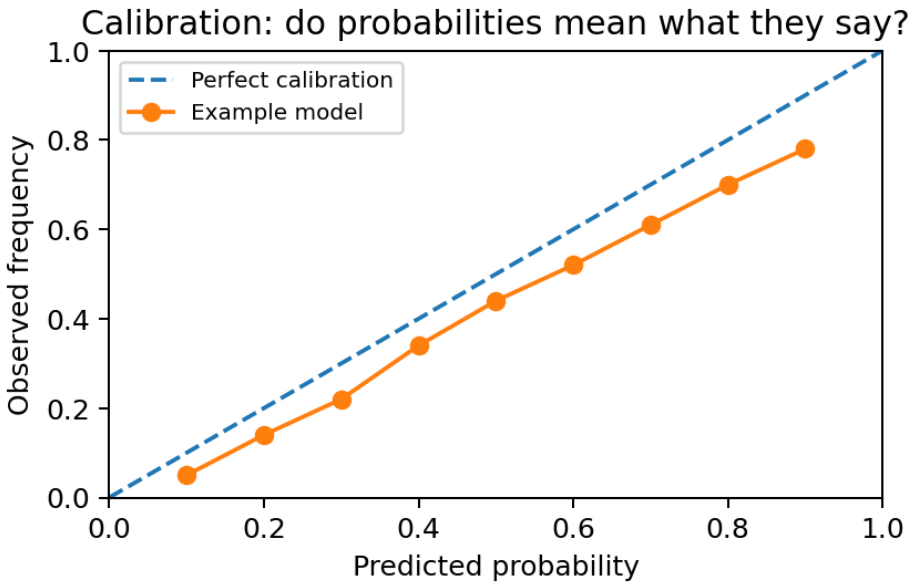
Key distinction - Model metrics tell you whether the model predicts/ranks/calibrates well. Product metrics tell you whether the game outcome improves. A strong offline metric does not guarantee better retention or revenue.

Use case	Useful metrics	What they answer	Watch out
Balanced binary classification	Accuracy, Precision, Recall, F1, ROC-AUC	Classification quality / ranking	Accuracy can hide class-specific failures.
Imbalanced classification (e.g. rare churn/event)	PR-AUC, Recall, Precision@target recall, F1, MCC	Can we find rare positives without too many false alarms?	ROC-AUC may look strong even when positive precision is poor.
Probability prediction / risk	Log Loss, Brier Score, Calibration / ECE	Are probabilities sharp and trustworthy?	AUC can be high while probabilities are badly calibrated.
Multiclass classification	Macro-F1, Micro-F1, weighted F1, Log Loss, OvR ROC-AUC	Performance across several classes	Macro exposes weak minority classes; micro is dominated by common classes.
Regression	MAE, RMSE, R^2	How close are numeric predictions?	RMSE punishes large errors more; MAPE breaks near zero.
Ranking / recommendation	NDCG@K, MAP@K, MRR, Recall@K, HitRate@K	Are relevant items near the top?	Offline ranking does not prove causal product impact.
Contextual bandit / RL	Mean reward, cumulative reward, cumulative regret, policy value	Does the policy earn reward while learning?	Reward design can create bad incentives.
Causal / uplift	ATE/CATE, AUUC, Qini, uplift@K	Who changes because of treatment?	Prediction accuracy is not causal effect quality.
Game/product outcome	D1/D7/D30 retention, conversion, session depth, progression, revenue + guardrails	Did the player experience/business improve?	Always pair target KPI with safety/fairness/frustration guardrails.

ROC-AUC, PR-AUC and calibration

- ROC curve: TPR vs FPR over all thresholds. ROC-AUC = probability a randomly chosen positive is ranked above a randomly chosen negative.
- PR curve: Precision vs Recall. PR-AUC is often more informative when positives are rare.
- Calibration: among predictions near 0.70, roughly 70% should occur if probabilities are calibrated.





Precision = TP / (TP + FP)

Recall = TPR = TP / (TP + FN)

F_1 = 2 · Precision · Recall / (Precision + Recall)

FPR = FP / (FP + TN)

Brier = (1/N) Σ_i $(\hat{p}_i - y_i)^2$

RMSE = $\sqrt{[(1/N)\ \Sigma_i\ (\hat{y}_i - y_i)^2]}$

MAE = (1/N) Σ_i $|\hat{y}_i - y_i|$

7A. ROC, PR and Calibration - Deep Dive

These three ideas answer different questions. ROC-AUC measures ranking quality, Precision-Recall focuses on finding positives without too many false alarms, and calibration asks whether predicted probabilities mean what they claim to mean.

ROC curve - TPR versus FPR

A binary classifier usually outputs a score or probability. A threshold converts that score into a class decision. Moving the threshold changes the confusion matrix, so ROC evaluates the model across all thresholds rather than choosing only one.

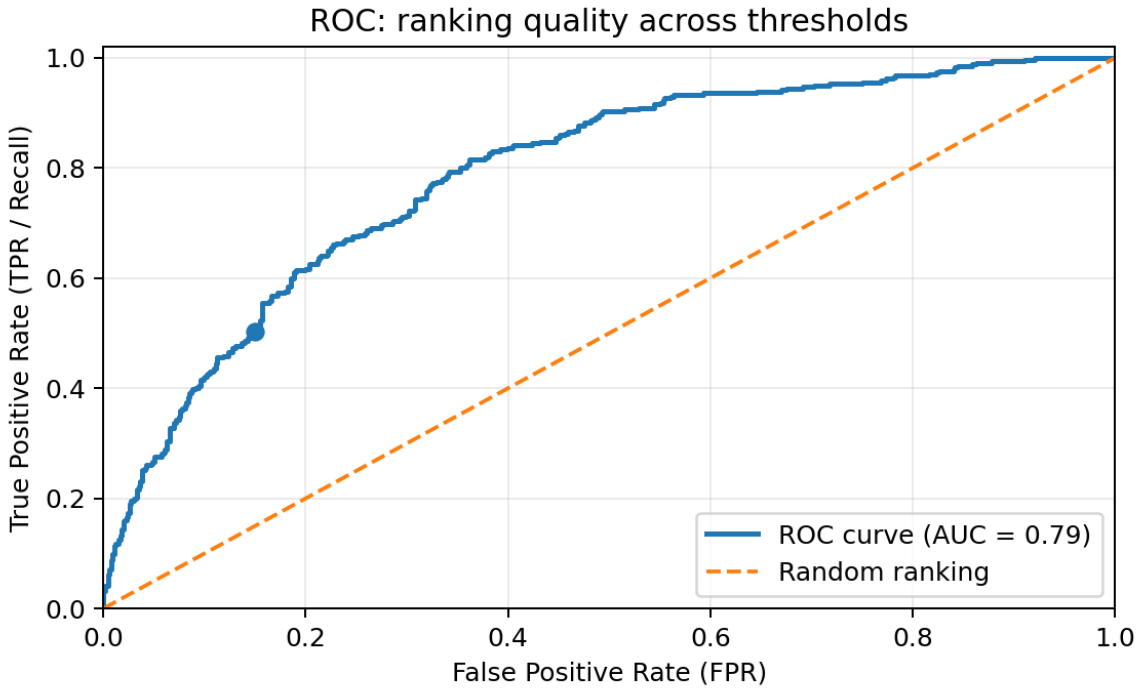
TPR = Recall = TP / (TP + FN)

FPR = FP / (FP + TN)

Example: 1,000 players, of whom 100 actually churn and 900 do not. The same model can behave very differently depending on the threshold:

Threshold	TP	FN	FP	TN	TPR / Recall	FPR	Precision
0.80	45	55	18	882	45%	2%	71.4%
0.50	75	25	90	810	75%	10%	45.5%
0.20	95	5	360	540	95%	40%	20.9%

Lowering the threshold catches more true churners (higher TPR/Recall), but it also flags more non-churners incorrectly (higher FPR). The ROC curve traces this trade-off.



Illustrative ROC curve. The diagonal is random ranking; better models bend toward the upper-left corner.

ROC-AUC interpretation

ROC-AUC is the area under the ROC curve. AUC = 0.5 corresponds to random ranking; AUC = 1.0 is perfect ranking. The most useful interview interpretation is:

ROC-AUC = P(score(positive) > score(negative))

So ROC-AUC = 0.85 means that if we randomly choose one positive and one negative example, the model ranks the positive above the negative about 85% of the time. It does NOT mean 85% accuracy.

Precision-Recall and PR-AUC

Precision asks: among the cases I predicted as positive, how many really are positive? Recall asks: among all real positives, how many did I find?

Precision = TP / (TP + FP)

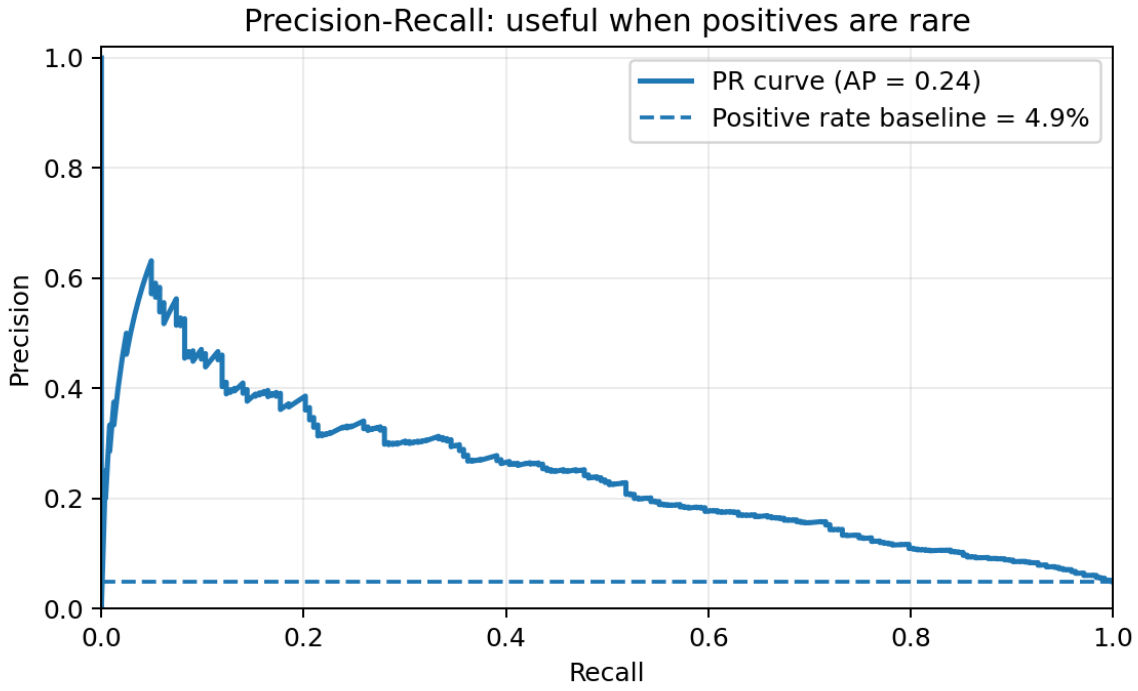
Recall = TP / (TP + FN)

Rare-positive example: 1,000 players, only 50 will churn. Suppose the model flags 100 players; 40 truly churn and 60 do not.

Precision = 40 / 100 = 40%

Recall = 40 / 50 = 80%

This model finds most churners, but 60% of its alerts are false positives. That matters if every intervention costs money, gives away game economy, or annoys players.



Illustrative PR curve with a 5% positive prevalence. The horizontal baseline is the positive rate.

Why PR-AUC is often better for rare positives

With extreme class imbalance, ROC can look strong because the denominator of FPR contains a huge number of true negatives. Precision directly exposes the cost of false positives. For fraud, rare churn, abuse, rare failures or medical screening, PR-AUC is often more revealing.

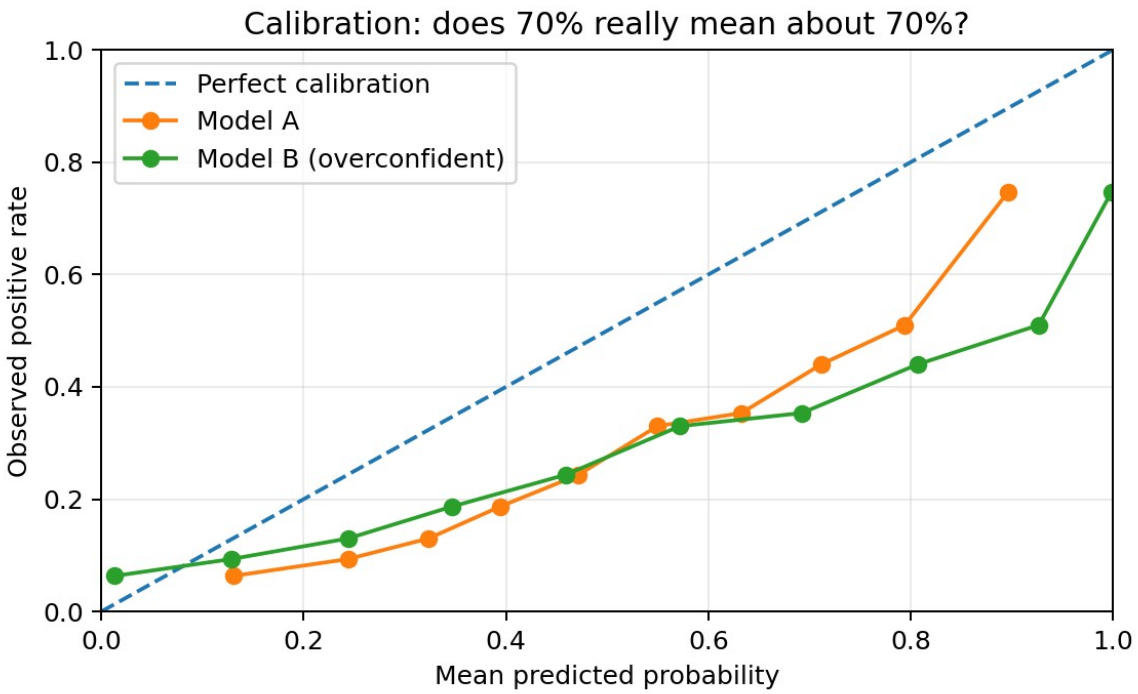
Rule of thumb: ROC-AUC answers “does the model rank positives above negatives?” PR-AUC answers “when I chase positives, how clean and complete are my alerts?”

Calibration - are the probabilities trustworthy?

Ranking and calibration are different. Two models can rank every player in the same order, and therefore have the same ROC-AUC, while assigning very different probabilities.

A calibrated model satisfies the intuition: among predictions near 0.70, roughly 70% of those cases should actually be positive.

Predicted probability	Observed positive rate	Interpretation
10%	11%	Well calibrated
30%	29%	Well calibrated
50%	51%	Well calibrated
70%	68%	Well calibrated
90%	89%	Well calibrated



Calibration plot. Perfect calibration lies on y = x. A curve below the diagonal means probabilities are too high (overconfident).

Why calibration matters for decisioning

Imagine a rule: if churn risk > 80%, give the player 500 coins. If the model predicts 85% but only 45% of such players actually churn, the ranking may still be useful, but the probability is not trustworthy and the intervention budget will be badly allocated.

Useful probability-quality metrics:

Brier = $(1/N)\ \Sigma_i\ (\hat{p}_i - y_i)^2$

Lower Brier is better. It is the mean squared error of predicted probabilities.

LogLoss = $-(1/N)\ \Sigma_i\ [\ y_i \log(\hat{p}_i) + (1-y_i) \log(1-\hat{p}_i)\ ]$

Lower Log Loss is better. It strongly penalizes predictions that are extremely confident and wrong.

Python - ROC, PR and calibration

```
from sklearn.metrics import (
   roc_auc_score, roc_curve,
   average_precision_score, precision_recall_curve,
   brier_score_loss, log_loss
)
from sklearn.calibration import calibration_curve

# y_true: 0/1 outcomes; y_prob: predicted P(y=1)
roc_auc = roc_auc_score(y_true, y_prob)
fpr, tpr, roc_thresholds = roc_curve(y_true, y_prob)

pr_auc = average_precision_score(y_true, y_prob)
precision, recall, pr_thresholds = precision_recall_curve(y_true, y_prob)

brier = brier_score_loss(y_true, y_prob)
ll = log_loss(y_true, y_prob)
```

```
observed_rate, mean_pred = calibration_curve(
    y_true, y_prob, n_bins=10, strategy="quantile"
)
```

Interview answer - concise version

"I would not evaluate a churn model with one metric. ROC-AUC tells me how well it ranks churners above non-churners. If churn is rare, I care strongly about PR-AUC because false positives become more visible. If predicted probabilities feed a decision policy, I also check calibration, Brier score or log loss. Finally, offline model quality is not enough: I validate whether using the model improves the product metric in a controlled online experiment."

8. Dynamic Difficulty Adjustment and RL

- Use the skill/difficulty posterior to estimate P(success) for a proposed level configuration.
- Adjust only permitted parameters; keep changes gradual and bounded by product constraints.
- Optimize a product objective with guardrails, not simply "make success probability 50%".

When RL? - Use full RL when actions have important sequential, long-horizon consequences. If decisions are mostly one-step with short-horizon reward, contextual bandits or supervised models are easier to operate and evaluate.

$$Q^*(s,a) = E[\ \Sigma_t\ \gamma^t\ r_t \mid s_0=s,\ a_0=a,\ \pi^*\ ]$$

9. Production / MLOps Terms

Term	Meaning
TTL	Time To Live: how long a feature/cache/decision remains valid.
SLI	Measured indicator, e.g. p95 latency.
SLO	Reliability target, e.g. p95 < 100 ms, availability ≥ 99.9%.
SLA	External/contractual commitment.
Canary	Expose a small % first to detect operational/product risk.
Shadow	Run new policy without affecting users; compare decisions offline.
Fallback	Safe deterministic/base action on timeout/failure.
Feature freshness	Maximum acceptable age of a feature.
Policy versioning	Exact model/policy ID logged for each decision.

10. Experience Narrative - How to Tell Your Story

Positioning - Your story is the combination of classical ML + adaptive decisioning + game telemetry/experimentation + modern AI infrastructure, not "LLM engineer learning personalization."

- Hegemon: feature experiments during game iteration/soft launch; D1/D7 retention; randomized control/treatment; effect size + confidence intervals; safe rollout.
- E-learning: contextual decisioning/bandit pattern; student context → candidate content → action → reward → policy update.
- Classical ML career: classification, regression, feature engineering, predictive modeling and evaluation.
- Fortis: live game telemetry, release comparisons, player KPIs, AI infrastructure, orchestration, observability and production systems.

Bridge sentence - "I have used these pieces in different domains: classical predictive ML, adaptive decisioning in education, and experimentation/telemetry in games. This role is about bringing those pieces together into one large-scale player decisioning platform."

11. Fast Interview Q&A

Design a decisioning system for Candy Crush.

> *State → features → candidates → policies/rules → arbitration → action → logging/feedback. Add latency SLOs, fallbacks, traceability and experimentation.*

How stop ML systems fighting each other?

Central arbitration with global objectives, priorities, mutual exclusions, budgets and hard product constraints.

A/B vs contextual bandit?

A/B estimates causal lift under controlled randomization; bandit adaptively chooses actions while balancing exploration/exploitation.

Why prediction not enough?

Prediction estimates likely outcomes under observed patterns; causal inference estimates what changes under intervention.

How know personalization is good?

Offline ranking/calibration + online controlled product impact against baseline + guardrails.

How estimate player skill?

Bayesian IRT/Rasch: jointly infer latent skill and level difficulty from outcomes, with uncertainty.

How evaluate new policy offline?

Replay/IPS/Doubly Robust on logged context-action-propensity-reward data; then validate online.

Would you use RL?

Only when long-horizon sequential consequences justify the complexity; otherwise prefer simpler methods.

12. Ultra-Quick Formula Reference

Concept	Formula
A/B uplift	$\hat{\Delta} = \hat{p}^B - \hat{p}^A$
SE of proportion	$SE(\hat{p}) = \sqrt{[\hat{p}(1-\hat{p})/n]}$
SE of difference	$SE(\hat{\Delta}) = \sqrt{[SE^{A2} + SE^{B2}]}$
z statistic	$z = \hat{\Delta} / SE(\hat{\Delta})$
95% CI	$\hat{\Delta} \pm 1.96 \cdot SE(\hat{\Delta})$
Two-sided p-value	$2 \cdot [1-\Phi(\lvert z \rvert)]$
ATE	$E[Y(1)-Y(0)]$
CATE	$E[Y(1)-Y(0) \mid X=x]$
LinUCB	$x^T\hat{\theta}_a + \alpha\sqrt{(x^T A_a^{-1} x)}$
Bayes	$p(\theta \mid D) = p(D \mid \theta)p(\theta)/p(D)$
Rasch	$P(\text{success}) = \sigma(\text{skill} - \text{difficulty})$
Brier	$(1/N)\Sigma(\hat{p}_i - y_i)^2$
RMSE	$\sqrt{[(1/N)\Sigma(\hat{y}_i - y_i)^2]}$

Mental shortcut - A/B = measure causally. Bandit = personalize while learning. Bayesian = update uncertain beliefs. OPE = estimate new policy before launch. Orchestrator = resolve competing decisions. Logging = make every decision reproducible and learnable.